

Estructuras Probabilísticas II

Bloom Filters y Count-Min Sketch

IIC2440 — Procesamiento de Datos Masivos

Clase 06

Agenda

1. Recap: funciones de hash
2. Bloom filters: motivación y construcción
3. Análisis probabilístico y k óptimo
4. Count-Min Sketch
5. Actividad: Postgres vs Bloom filter
6. Cierre y adelanto

De dónde venimos

Clase anterior: sampling. Trabajar con un subconjunto de los datos.

Hoy: hashing. Transformar **todo** el espacio de datos a un espacio mucho más pequeño, usando funciones que reparten "uniformemente".

Ambas estrategias son probabilísticas, pero responden preguntas distintas:

- Sampling → *¿cómo se ve el dataset en general?*
- Hashing → *¿está este elemento? ¿cuántas veces apareció?*

Recap: funciones de hash

Funciones de hash

Una **función de hash** h mapea inputs de tamaño arbitrario a palabras de b bits:

$$h : \mathcal{U} \rightarrow \{0, 1, \dots, 2^b - 1\}$$

- Determinista: mismo input $\rightarrow$ mismo output
- Rápida de computar
- Debe repartir "uniformemente" los outputs

Ejemplo trivial: $h(x) = x \bmod n$.

Por principio del palomar, si $|\mathcal{U}| > n$ hay **colisiones**: $h(m_1) = h(m_2)$ con $m_1 \neq m_2$.

Hash ideal

En esta unidad, asumiremos que nuestras funciones de hash son **ideales**:

$h : S \rightarrow D$ (con $|S| \gg |D|$) es un **hash ideal** si para $s \in S$ y $d \in D$ arbitrarios:

$$P(h(s) = d) = \frac{1}{|D|}$$

En la práctica esto es **casi cierto** para funciones como SHA, MurmurHash, xxHash. En Python:

```
import hashlib  
h = int(hashlib.sha256(b"jreutter@puc.cl").hexdigest(), 16) % n
```

Asumir idealidad es una **abstracción conveniente** — los análisis con hashes reales son solo aproximadamente ciertos, pero la aproximación es excelente.

Bloom filters

El problema

Nuestro sistema recibe **cientos de logins por segundo**. Por cada uno, necesitamos verificar:

¿Es `jreutter@puc.cl` un usuario registrado?

Tenemos $N = 10^9$ usuarios en una tabla indexada. Una consulta con índice cuesta $\sim \log N \approx 30$ operaciones.

Parece poco, pero:

- 30 ops × cientos de logins/seg × la mayoría **ni siquiera son usuarios reales** (bots, typos, ataques)
- Queremos **filtrar rápido** los que claramente no están, sin tocar la DB

La idea del Bloom filter

Reservamos un arreglo de bits B de tamaño n (ej: 1 GB $\approx 8 \times 10^9$ bits).

Construcción:

1. Inicializamos $B[i] = 0$ para todo i .
2. Para cada usuario real u : calculamos $h(u)$ y ponemos $B[h(u)] = 1$.

Consulta de un nuevo v :

- Si $B[h(v)] = 0 \rightarrow v$ **no está** en la base de datos (garantizado).
- Si $B[h(v)] = 1 \rightarrow v$ **podría estar**. Verificamos en la DB.

Cada consulta rápida = una operación de hash + un acceso a memoria.

¿Por qué funciona?

Si v está registrado: cuando lo agregamos a M , pusimos un 1 en $B[h(v)]$. Luego ese bit sigue en 1. La consulta siempre dice "podría estar". → **No hay falsos negativos.**

Si v no está registrado: $B[h(v)]$ podría ser 0 (lo rechazamos inmediatamente) o podría ser 1 si algún otro usuario u real colisionó con v en el hash. → **Sí hay falsos positivos.**

El output "podría estar" es la razón de todavía ir a consultar la DB. Pero el output "no está" es **definitivo**.

Definición formal

Un **Bloom filter** sobre un conjunto S tiene:

1. Un arreglo B de n bits.
2. Una colección $h_1, \dots, h_k$ de funciones de hash, cada una $S \rightarrow [0, n - 1]$.
3. Un conjunto $M \subseteq S$ de valores "permitidos".

Inserción de $v \in M$: poner $B[h_i(v)] = 1$ para todo $i \in \{1, \dots, k\}$.

Consulta de v : retornar "sí" si $B[h_i(v)] = 1$ para **todo** i ; "no" si algún $B[h_i(v)] = 0$.

La versión con una sola función (lo visto antes) es el caso $k = 1$.

Análisis probabilístico

Caso base: una función de hash

Setup: $N = |M|$ usuarios, n bits en el arreglo, una función h .

Para un bit fijo $B[i]$ y un usuario fijo $u \in M$:

$$P(h(u) = i) = \frac{1}{n} \implies P(h(u) \neq i) = 1 - \frac{1}{n}$$

Como los hashes son independientes (idealidad), después de insertar los N usuarios:

$$P(B[i] = 0) = \left(1 - \frac{1}{n}\right)^N$$

Aproximación con e

Usando la identidad $(1 - \epsilon)^{1/\epsilon} \approx e^{-1}$ para ϵ pequeño:

$$\left(1 - \frac{1}{n}\right)^N = \left(1 - \frac{1}{n}\right)^{n \cdot N/n} \approx e^{-N/n}$$

Entonces, para $v \notin M$:

$$P(\text{falso positivo con } k = 1) = P(B[h(v)] = 1) \approx 1 - e^{-N/n}$$

Ejemplo. $N = 10^9, n = 8 \times 10^9$: FP $\approx 1 - e^{-1/8} \approx 0,118$.

Es decir, el **88%** de los usuarios fake se descartan sin tocar la DB. Buen comienzo.

Y cómo podemos mejorar esto?

Queremos disminuir la probabilidad de un falso positivo, pues cada uno de ellos nos hace ir a la base de datos de forma innecesaria.

k funciones de hash

Agreguemos k funciones independientes. Cada una "marca" un bit por usuario.

Para un bit fijo después de insertar N usuarios, con k funciones cada uno:

$$P(B[i] = 0) = \left(1 - \frac{1}{n}\right)^{kN} \approx e^{-kN/n}$$

Para un $v \notin M$, el filtro dice "sí" solo si **todos** los k bits $B[h_i(v)]$ son 1:

$$P(\text{falso positivo}) \approx \left(1 - e^{-kN/n}\right)^k$$

¿Cuál es el k óptimo?

$P(\text{FP}) \approx (1 - e^{-kN/n})^k$. Hay una tensión:

- k muy pequeño $\rightarrow$ pocos bits a verificar $\rightarrow$ colisiones son "fáciles"
- k muy grande $\rightarrow$ el arreglo se satura de 1s $\rightarrow$ casi todo da "sí"

Derivación. Sea $q = e^{-kN/n}$ la fracción de bits que **siguen en 0**. Entonces $k = (n/N) \ln(1/q)$ y:

$$\ln P(\text{FP}) = k \ln(1 - q) = \frac{n}{N} \ln(1/q) \ln(1 - q)$$

Minimizamos $g(q) = \ln(1/q) \ln(1 - q) = -\ln q \cdot \ln(1 - q)$.

Derivación del k óptimo

Derivando $g(q) = -\ln q \cdot \ln(1 - q)$ e igualando a cero:

$$g'(q) = -\frac{\ln(1 - q)}{q} + \frac{\ln q}{1 - q} = 0 \iff (1 - q) \ln(1 - q) = q \ln q$$

Por simetría, la solución es $q = 1/2 \rightarrow$ **el arreglo óptimo está medio lleno.**

De $q = e^{-kN/n} = 1/2$ despejamos:

$$k^* = \frac{n}{N} \ln 2 \approx 0,693 \cdot \frac{n}{N}$$

y sustituyendo en $P(\text{FP})$:

$$P(\text{FP})^* = (1 - q)^{k^*} = \left(\frac{1}{2}\right)^{k^*} \approx 0,6185^{n/N}$$

Ejemplo numérico

$N = 10^9$ usuarios, $n = 8 \times 10^9$ bits (1 GB), así que $n/N = 8$ y $k^* \approx 5,5$:

k	$P(\text{FP}) \approx (1 - e^{-k/8})^k$
1	0,118
2	0,049
3	0,031
5	0,022
6 ($\approx k^*$)	0,022
10	0,034
20	0,180

Moraleja: con $k \approx 5$ -6 hashes y 1 GB de bits, solo $\sim 2\%$ de los fakes llega a la DB. Pasarse de k empeora (el arreglo se satura).

Resumen práctico

Para diseñar un Bloom filter:

1. Elijo el FP máximo que acepto: ε .
2. Calculo bits por elemento: $n/N = -\log_2 \varepsilon / \ln 2 \approx 1,44 \log_2(1/\varepsilon)$.
3. Calculo k óptimo: $k^* = (n/N) \ln 2$.

Reglas de bolsillo:

ε	Bits por elemento	k óptimo
10%	5	3
1%	10	7
0,1%	15	10
0,01%	20	14

Count-Min Sketch

Otro problema distinto

El Bloom filter responde: "*¿está este elemento?*".

A veces queremos: "*¿cuántas veces apareció este elemento?*".

Ejemplos:

- Top productos más consultados en una API
- IPs que más piden (detección de abuso)
- Palabras más frecuentes en un stream de tweets

Si tengo billones de eventos, un contador exacto por clave no cabe en memoria.

La construcción

Un **Count-Min Sketch** con parámetros d (filas) y w (columnas):

- Matriz C de enteros de $d \times w$, inicializada en 0.
- d funciones de hash independientes $h_1, \dots, h_d : \mathcal{U} \rightarrow [0, w - 1]$.

Incremento cuando vemos el evento x :

```
para i en 1..d:  
    C[i][h_i(x)] += 1
```

Consulta del conteo aproximado de x :

```
retornar min_{i} C[i][h_i(x)]
```

¿Por qué **min**?

Cada celda $C[i][h_i(x)]$ acumula x y **todas las colisiones** de x con otros elementos en la fila i .

Entonces $C[i][h_i(x)] \geq \text{conteo real}(x)$ siempre: **nunca subestima**.

Las distintas filas tienen hashes independientes, así que el set de colisiones cambia.

Tomando el **min** nos quedamos con la fila que *menos* colisionó con x .

Propiedad clave: $\hat{f}(x) \geq f(x)$ siempre. El error es **unilateral** (sobreestima, nunca subestima).

Análisis — error esperado

Sea $f(x)$ el conteo real y $N = \sum_y f(y)$ el total de eventos. En cada fila i :

$$C[i][h_i(x)] = f(x) + X_i, \quad X_i = \sum_{y \neq x} f(y) \cdot \mathbb{1}[h_i(y) = h_i(x)]$$

X_i es la **contaminación** de x por colisiones en la fila i . Asumimos que h es ideal, luego $P(h_i(y) = h_i(x)) = 1/w$.

Por linealidad de la esperanza:

$$E[X_i] = \sum_{y \neq x} \frac{f(y)}{w} = \frac{N - f(x)}{w} \leq \frac{N}{w}$$

En promedio, cada fila sobreestima por a lo más N/w . Pero queremos un **bound con alta probabilidad**.

Análisis — garantía

Por Markov:

$$P(X_i \geq \varepsilon N) \leq \frac{E[X_i]}{\varepsilon N} \leq \frac{1}{\varepsilon w}$$

Eligiendo $w = \lceil e/\varepsilon \rceil$: $P(X_i \geq \varepsilon N) \leq 1/e$.

Como las d filas usan hashes independientes, $\hat{f}(x) = f(x) + \min_i X_i$ supera εN solo si **todas** las filas lo hacen:

$$P(\hat{f}(x) - f(x) \geq \varepsilon N) \leq (1/e)^d$$

Eligiendo $d = \lceil \ln(1/\delta) \rceil$:

$$P(\hat{f}(x) \leq f(x) + \varepsilon N) \geq 1 - \delta$$

Ejemplo numérico

$\varepsilon = 0,001, \delta = 0,01$:

- $w = \lceil e/0,001 \rceil = 2719$
- $d = \lceil \ln(100) \rceil = 5$
- Memoria: $\sim 14\,000$ enteros ≈ 56 KB

Sin importar cuántas claves distintas vengan en el stream, la estructura pesa lo mismo.

Con $N = 10^9$ eventos procesados, el error está acotado por $\varepsilon N = 10^6$ con probabilidad 99%. Útil cuando buscamos **elementos frecuentes** (heavy hitters), no cuando buscamos frecuencias raras.

Count-Min es la "contraparte frecuencista" del Bloom filter: misma idea (memoria fija + hashes), pregunta distinta.

Limitaciones que conviene tener presentes

Bloom filter:

- No se puede **eliminar** (borraría bits compartidos). *Solución: counting Bloom filter* — bits $\rightarrow$ contadores.
- No se puede **enumerar** los elementos guardados.
- N debe estimarse de antemano para fijar n y k .

Count-Min sketch:

- Sobreestima cuando hay muchas claves de baja frecuencia (la "cola" contamina las celdas).
- *Variante: Count Sketch*, con signos $\pm 1 \rightarrow$ estimación insesgada.

Mención: HyperLogLog

Una tercera estructura del mismo espíritu, para una pregunta distinta:

¿Cuántos elementos distintos vimos en el stream?

Idea: mantener el máximo número de ceros iniciales que haya producido $h(x)$ en binario. Si vimos ρ ceros, hay del orden de 2^ρ elementos distintos.

- Memoria: $\sim 1,5$ KB.
- Error relativo $\sim 1\%$ contando billones de elementos distintos.
- Lo usan Redis (`PFCOUNT`), Presto, BigQuery (`APPROX_COUNT_DISTINCT`).

No alcanzamos a ver el análisis hoy — queda como lectura recomendada.

Actividad práctica

Postgres vs Bloom filter

Objetivo

Implementar un Bloom filter desde cero y compararlo con Postgres para el problema de **validar usernames en bulk**.

Vamos a medir:

1. **Tiempo de setup** (inserción de N usernames en cada sistema).
2. **Tiempo de query** sobre Q consultas (mezcla de reales y fakes).
3. **Tasa de falsos positivos** empírica del Bloom filter.
4. **Tiempo combinado** de una estrategia "Bloom primero, luego Postgres".

Parte 0 — Generar el dataset

Vamos a trabajar con $N = 10^7$ usernames sintéticos. En `setup.py`:

```
import uuid, csv

N = 10_000_000
with open("users.csv", "w") as f:
    w = csv.writer(f)
    for _ in range(N):
        w.writerow([str(uuid.uuid4())])
```


Parte 1 — Cargar en Postgres

```
CREATE TABLE users (username TEXT PRIMARY KEY);  
\COPY users FROM 'users.csv' CSV;  
ANALYZE users;
```

El `PRIMARY KEY` ya crea un índice B-tree sobre `username`, así que las consultas `WHERE username = ?` van por índice.

Parte 2 — Diseño del Bloom filter

Truco clásico (**Kirsch–Mitzenmacher**): basta con **dos** hashes "buenos" para simular k funciones:

$$h_i(x) = (h_a(x) + i \cdot h_b(x)) \bmod n$$

Esto evita calcular k hashes SHA-256 por inserción/consulta — basta partir un solo hash de 256 bits en dos mitades.

Parámetros sugeridos: $\varepsilon = 0,01 \Rightarrow n \approx 10N = 10^8$ bits (12,5 MB), $k = 7$.

Parte 2 — Implementación

```
import hashlib
from bitarray import bitarray

class BloomFilter:
    def __init__(self, n_bits, k):
        self.n, self.k = n_bits, k
        self.B = bitarray(n_bits); self.B.setall(0)
    def _hashes(self, x):
        d = hashlib.sha256(x.encode()).digest()
        ha = int.from_bytes(d[:8], "big")
        hb = int.from_bytes(d[8:16], "big")
        return [(ha + i * hb) % self.n for i in range(self.k)]
    def add(self, x):
        for j in self._hashes(x): self.B[j] = 1
    def __contains__(self, x):
        return all(self.B[j] for j in self._hashes(x))
```

Parte 3 — Poblar y medir

```
import time

# Setup
bf = BloomFilter(n_bits=10**8, k=7)
t0 = time.time()
for u in open("users.csv"):
    bf.add(u.strip())
print(f"Bloom setup: {time.time()-t0:.2f}s")
```

Comparemos con el tiempo que tomó `\COPY` en Postgres (suele ser más rápido por `COPY` nativo, pero Bloom no necesita disco).

Parte 4 — Benchmark: Postgres puro

Generar $Q = 10^6$ consultas: 50% usernames reales (muestreados del dataset), 50% UUIDs aleatorios nuevos (fakes).

```
import psycopg2, time

conn = psycopg2.connect(...)
cur = conn.cursor()

t0 = time.time()
for q in queries:
    cur.execute("SELECT 1 FROM users WHERE username=%s", (q,))
    cur.fetchone()
print(f"Postgres: {time.time()-t0:.2f}s")
```

Parte 4 — Benchmark: Bloom + Postgres

```
t0 = time.time()
fp_count = 0
for q in queries:
    if q in bf:                                     # filtro rápido
        cur.execute("SELECT 1 FROM users WHERE username=%s", (q,))
        if cur.fetchone() is None:
            fp_count += 1                           # falso positivo
print(f"Bloom + Postgres: {time.time()-t0:.2f}s, FPs: {fp_count}")
```

`fp_count` mide los falsos positivos **empíricos**: queries que pasaron el filtro pero no estaban en la DB.

Parte 5 — Análisis

Vamos a reportar:

1. **Tiempo puro Postgres vs Bloom + Postgres.** ¿Cuánto se ahorró?
2. **Tasa de FP empírica:** `fp_count / (número de queries fake)`. Compararla con la predicción teórica $(1 - e^{-kN/n})^k$.
3. **Memoria** usada por el Bloom ($n/8$ bytes) vs memoria del índice Postgres (`SELECT pg_size_pretty(pg_relation_size('users_pkey'));`).

Pregunta de discusión. Si solo el 5% de las consultas son usernames reales, ¿cambia la respuesta? ¿Y si es el 95%?

Lo que esperamos ver

- El Bloom filter solo responde "sí"/"no" en **memoria** → $\sim$ ns por consulta.
- Postgres responde desde un **B-tree** → típicamente μ s por consulta (cache hits) o ms (cache misses).
- Para consultas que **no están**, el Bloom las descarta sin siquiera tocar Postgres.
- El speedup del combinado aumenta mientras más alta sea la proporción de fakes en el tráfico.

Por eso muchos sistemas reales (Cassandra, LevelDB, RocksDB) **incluyen Bloom filters por defecto** para evitar búsquedas innecesarias en disco.

Cierre

Lo que vimos hoy

- **Hash ideal** como abstracción de trabajo.
- **Bloom filter**: set membership probabilístico. Sin falsos negativos; falsos positivos controlables con k funciones y tamaño n .
- k **óptimo** $= (n/N) \ln 2$, con $FP \approx 0,6185^{n/N}$.
- **Count-Min Sketch**: frecuencias aproximadas con error unilateral.
- **HyperLogLog** (mención): conteo de distintos con memoria casi constante.

Patrón común: **memoria fija + hashes independientes** = respuesta aproximada con garantías controlables.

Próxima clase

Locally-Sensitive Hashing (LSH) — hashing que preserva **distancia**.

- Elementos cercanos → mismo hash (probable)
- Elementos lejanos → distinto hash (probable)

Esto cambia el juego: en vez de "¿está?", podemos preguntar "**¿qué hay cerca?**".

→ Bases de datos de vectores, búsqueda por similitud, el motor detrás de RAG y los sistemas de recomendación modernos.

Fin

¿Preguntas?